

좌표계와 좌표변환

안테나가 켜 각도가 지도 위 위경도가 되기까지

레이다 시스템 소프트웨어 스터디 · Part 2

발표 대본

표지 · 약 20초

안녕하십니까. 이번 과제의 주제는 좌표계와 좌표변환입니다. 레이다 안테나가 켜 거리와 각도가 전시기의 위도 · 경도가 되기까지, 그 사이에 어떤 좌표계를 거치고 각 단계에서 무엇이 바뀌는지를 다루겠습니다.

진행은 개념, 설계, C 구현 검증 순서입니다.

INDEX

목차

01 좌표계 여섯 가지

02 실전 설계

03 C 로 확인

2

발표 대본

목차 · 약 30초

세 단계로 진행하겠습니다.

첫째, 좌표계 여섯 가지입니다. 안테나, 동체, INS, NED/ENU, ECEF/ECI, LLA 각각 원점이 어디이고 축이 어디를 향하는지, 왜 필요한지를 정리합니다.

둘째, 실전 설계입니다. 함선에 고정된 안테나를 놓고 5단계 변환 경로를 설계하고, 각 단계에서 무엇이 회전과 이동을 공급하는지 봅니다.

셋째, C 로 확인입니다. 설계한 변환을 구현하고 왕복 시험과 확인 실험 세 가지로 숫자로 검증하고, UI 에서 자세를 바꿔 가며 확인합니다.

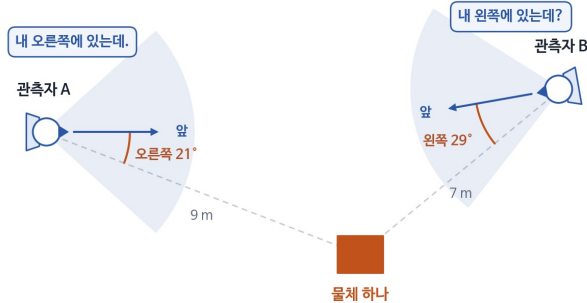
사전 지식은 삼각함수와 행렬 곱셈 정도면 충분하고, 그마저도 필요한 곳에서 다시 설명하겠습니다.

왜 좌표계가 여럿인가

같은 물체라도 어디에 서서 어디를 보느냐에 따라 말이 달라집니다

같은 물체를 두 사람이 본다

물체도 그 자리, 두 사람도 그 자리. 달라진 것은 어디에 서서 어디를 보느냐뿐이다.



관측자 A 가 말하면

"내 오른쪽에 있는데."

앞에서 오른쪽으로 21°, 9 m

관측자 B 가 말하면

"내 왼쪽에 있는데?"

앞에서 왼쪽으로 29°, 7 m

물체는 하나 · 숫자는 둘

달라진 것은 두 가지뿐입니다 — 어디에 서서 재는가 (원점), 어디를 보고 재는가 (축).

좌표계란 그 둘을 미리 정해 놓은 약속이고, 기준이 다른 두 좌표계의 숫자를 서로 옮겨 주는 계산이 좌표변환입니다.

이 발표에서는 같은 일이 "레이다가 잰 각도" 와 "전시기의 위·경도" 사이에서 일어납니다.

3

발표 대본

개념 · 약 60초

그림의 물체는 하나입니다. 그런데 그 물체를 보는 사람이 둘입니다. 왼쪽 관측자 A 와 오른쪽 관측자 B 는 서 있는 자리도 다르고 보고 있는 방향도 다릅니다.

A 에게 물체는 자기 앞에서 오른쪽으로 21도, 9 미터 거리에 있습니다. 같은 물체가 B 에게는 자기 앞에서 왼쪽으로 29도, 7 미터입니다. 한쪽은 오른쪽이라 하고 다른 쪽은 왼쪽이라 합니다. 둘 다 맞는 말입니다.

물체도 그 자리에 있고 두 사람도 그 자리에 있습니다. 달라진 것은 두 가지뿐입니다. 어디에 서서 재는가, 그리고 어디를 보고 재는가. 앞의 것을 원점, 뒤의 것을 축이라고 부릅니다.

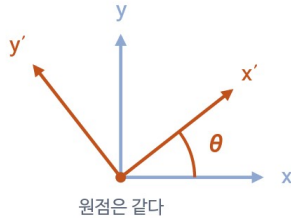
그래서 좌표계란 어디에 서서 어느 방향을 기준으로 재는가를 미리 정해 놓은 약속입니다. 기준이 여럿이면 같은 점의 숫자도 여럿이 되고, 그 숫자를 서로 옮겨 주는 계산이 좌표변환입니다.

이 발표에서는 같은 일이 레이다와 전시기 사이에서 일어납니다. 레이다는 안테나 정면을 기준으로 각도를 재고, 전시기는 지구 전체를 기준으로 위도와 경도를 씁니다. 그 사이를 메우는 것이 오늘 이야기의 전부입니다.

좌표변환의 기본 동작 — 회전과 평행이동

복잡한 수학처럼 들리지만 실제로 하는 일은 두 가지입니다

① 회전 (rotation) — 기준 방향이 다를 때
두 좌표계의 축이 어긋난 만큼 θ 돌려서 맞춘다

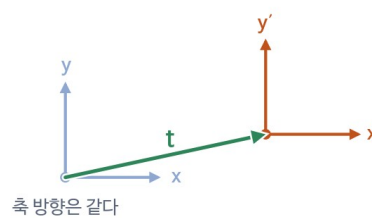


원점은 같다

① 회전 — 방향만 바꾼다

길이도 각도도 변하지 않는다. 축을 돌려 다시 읽을 뿐이다.

② 평행이동 (translation) — 기준 점이 다를 때
두 좌표계의 원점이 떨어진 만큼 t 옮겨서 맞춘다



축 방향은 같다

② 평행이동 — 자리만 옮긴다

방향은 그대로다. 원점 사이의 차이를 벡터 하나로 더하거나 뺀다.

$$\text{새 좌표} = R \cdot \text{옛 좌표} + t$$

R 회전행렬 (3×3) · t 평행이동 (3×1). 모든 단계는 이 둘 중 하나거나, 둘 다입니다.

4

발표 대본

개념 · 약 40초

복잡한 수학처럼 들리지만 실제로 하는 일은 두 가지뿐입니다.

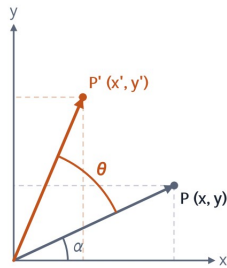
하나는 회전입니다. 두 좌표계의 축이 어긋나 있을 때 어긋난 각만큼 축을 돌려서 방향을 맞춥니다. 물체를 돌리는 것이 아니라 축을 돌려 다시 읽는 것이라, 길이도 각도 사이의 관계도 그대로입니다.

다른 하나는 평행이동입니다. 두 좌표계의 원점이 떨어져 있을 때 떨어진 만큼 옮겨서 자리를 맞춥니다. 이때 방향은 건드리지 않습니다.

그래서 모든 변환이 한 줄로 적힙니다. 새 좌표는 회전행렬 R 에 옛 좌표를 곱하고 평행이동 t 를 더한 것입니다. 뒤에 나올 다섯 단계도 전부 이 둘 중 하나거나 둘 다입니다. 그게 전부입니다.

회전행렬 — $R_z \cdot R_y \cdot R_x$

외울 것이 아니라 삼각함수 덧셈정리 한 줄에서 바로 나옵니다



원래 점을 극좌표로 쓰면

$$x = r \cdot \cos \alpha, \quad y = r \cdot \sin \alpha$$

 θ 만큼 더 돌리면 각이 $\alpha + \theta$ 가 되므로

$$x' = r \cdot \cos(\alpha + \theta) = r \cdot \cos \alpha \cdot \cos \theta - r \cdot \sin \alpha \cdot \sin \theta$$

$$y' = r \cdot \sin(\alpha + \theta) = r \cdot \sin \alpha \cdot \cos \theta + r \cdot \cos \alpha \cdot \sin \theta$$

 $r \cdot \cos \alpha = x, \quad r \cdot \sin \alpha = y$ 를 되돌려 넣으면

$$x' = x \cdot \cos \theta - y \cdot \sin \theta$$

$$y' = x \cdot \sin \theta + y \cdot \cos \theta$$

이 두 줄을 표로 적은 것이 $R_z(\theta)$ 다.

세 축 둘레의 회전행렬

 $R_z(\theta)$ — z 축 둘레 (yaw)

$$\begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

 $R_y(\theta)$ — y 축 둘레 (pitch)

$$\begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}$$

 $R_x(\theta)$ — x 축 둘레 (roll)

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix}$$

원리는 한 줄입니다 — 삼각함수 덧셈정리. 거기서 나온 두 줄에서 x, y 글자를 떼고 앞의 숫자만 표로 적은 것이 회전행렬입니다. $R_z \cdot R_y \cdot R_x$ 는 각각 $z \cdot y \cdot x$ 축 둘레의 회전입니다. 도는 축의 성분은 변하지 않으니 그 행과 열은 1 과 0 으로 남고, 나머지 두 축 자리에 위의 두 줄이 들어갑니다.되돌리기는 $\theta \rightarrow -\theta$ 이고 그것은 $\sin$ 의 부호만 바꾸는 것이라, 행과 열을 맞바꾼 것과 같습니다. $R^{-1} = R^T$

5

발표 대본

개념 · 약 60초

회전행렬이 어디서 나오는지만 보겠습니다. 왼쪽 그림입니다. 점 하나를 극좌표로 쓰고 각을 θ 만큼 더한 다음 삼각함수 덧셈정리를 펴면 두 줄이 남습니다. 새 x 는 $\cos \theta$ 곱하기 x 빼기 $\sin \theta$ 곱하기 y , 새 y 는 $\sin \theta$ 곱하기 x 더하기 $\cos \theta$ 곱하기 y 입니다.

이 두 줄에서 x, y 글자를 떼고 앞의 숫자만 표로 적은 것이 회전행렬입니다. 외울 것이 아니라 덧셈정리 한 줄에서 바로 나오는 것입니다.

오른쪽이 세 축 둘레의 회전행렬입니다. R_z 는 z 축 둘레로 도는 것이라 z 성분이 변하지 않습니다. 그래서 셋째 행과 셋째 열은 0, 0, 1 로 남고 나머지 x, y 자리에 방금 만든 두 줄이 그대로 들어갑니다. R_y 와 R_x 도 같은 방식입니다. 도는 축 자리만 1 로 남기고 나머지 두 축에 같은 모양을 넣습니다. R_y 만 부호가 반대쪽에 붙는데, 축을 x, y, z 순환 순서로 세면 y 축 둘레는 z 에서 x 로 도는 차례라서 그렇습니다.

쓸 때는 이 셋을 곱해서 씁니다. 동체에서 NED 로 갈 때 $R_z(\text{yaw})$ 곱하기 $R_y(\text{pitch})$ 곱하기 $R_x(\text{roll})$ 순서로 곱하는 것이 그것입니다.

하나만 더 기억해 두시면, 되돌릴 때 역행렬을 계산하지 않습니다. θ 를 $-\theta$ 로 바꾸면 $\sin$ 의 부호만 바뀌는데 그것이 곧 행과 열을 맞바꾼 것이라, R 의 역행렬은 R 의 전치입니다. 역변환 장에서 이 성질을 그대로 씁니다.

01

좌표계 여섯 가지

1 안테나 좌표계 — 극좌표 · 직교좌표 · UV

2 동체 (Body) 좌표계

3 INS 좌표계

4 NED / ENU 국지수평 좌표계

5 ECEF / ECI 지구 중심 좌표계

6 LLA 측지 좌표계

7 변환 체인

이 발표의 약속 : 안테나 · 동체 · NED 좌표계에서 z 는 아래 . "10 m 위" 는 $z = -10$ 입니다 .

6

발표 대본

장절 · 약 30초

Part 1 에서는 좌표계 여섯 가지를 하나씩 봅니다. 각 좌표계마다 원점이 어디이고, 축이 어디를 향하고, 무엇에 붙어 있는지를 확인하는 것이 핵심입니다. 안테나, 동체, INS, 국지수평 NED, 지구 중심 ECEF, 그리고 위경도 LLA 순서로 가고, 마지막에 이 여섯을 잇는 변환 체인에서 회전과 이동을 무엇이 공급하는지 정리합니다.

01. 안테나 좌표계 ① 극좌표 (Polar)

레이다가 물리적으로 잴 수 있는 것 — 거리 하나와 각도 둘. 기준은 안테나 자기 정면 (보어사이트) 입니다

측정

안테나

동체

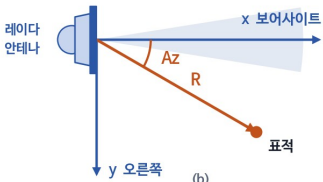
NED

ECEF

LLA

(a) 위에서 본 그림 — 방위각 Az (Azimuth)

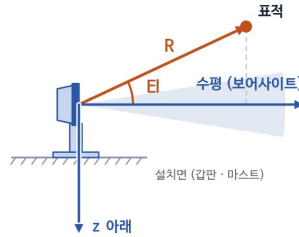
안테나를 위에서 내려다본 모습. 보어사이트에서 오른쪽으로 잴 각



원점은 안테나 배열면 한가운데.
(b)는 같은 안테나를 화살표 방향에서 본 모습이다.

(b) 옆에서 본 그림 — 고각 El (Elevation)

같은 안테나를 옆에서 본 모습. 수평면에서 위로 잴 각



z는 아래가 +. 표적이 수평면보다 위에 있으면 z는 음수가 된다.

세 성분

R 시선거리

Range · 전파 왕복시간 $\times c \div 2$

Az 방위각

Azimuth · 보어사이트에서 오른쪽으로

El 고각

Elevation · 수평면에서 위로

20 km / 30° / 0°

보어사이트 = 안테나가 똑바로 바라보는 방향. Az · El 은 전부 이 방향을 0 으로 놓고 잴 각이라, 배가 어느 쪽을 향하는지는 아직 들어오지 않습니다.

7

발표 대본

좌표계 · 약 55초

레이다가 물리적으로 잴 수 있는 것은 거리 하나와 각도 둘입니다.

R 은 시선거리로 전파 왕복시간 곱하기 광속 나누기 2 입니다. Az 는 방위각으로 보어사이트에서 오른쪽으로 잴 각, El 은 고각으로 수평면에서 위로 잴 각입니다. 과제 예제값이 20 km, 30도, 0도입니다.

보어사이트는 안테나가 똑바로 바라보는 방향입니다. 안테나 신호가 가장 세게 나가고 가장 세게 들어오는 한가운데 방향입니다.

그림에 배를 그리지 않은 이유를 짚고 가겠습니다. 극좌표는 안테나가 자기 정면을 기준으로 잴 값입니다. 배를 같이 그리면 선수와 선미가 먼저 눈에 들어와서 Az 를 배 기준 각으로 읽게 됩니다. 이 단계에 배는 아직 등장하지 않습니다. 배 기준으로 옮기는 일은 안테나에서 동체로 가는 단계에서 합니다.

원점은 안테나 배열면 한가운데이고 z 는 아래가 양수입니다. 그래서 표적이 수평면보다 위에 있으면 z 는 음수가 됩니다.

01. 안테나 좌표계 ② 직교좌표 (Cartesian)

같은 표적을 각도 대신 세 축 방향 거리 (x · y · z) 로 적는 것. 회전행렬은 벡터에만 곱할 수 있어서 이 표기가 필요합니다

측정

안테나

동체

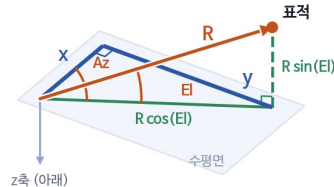
NED

ECEF

LLA

(a) 표적 하나를 세 축에 나눠 담기

x 보여사이트 · y 오른쪽 · z 아래. R을 El로, 그다음 Az로 나눈다



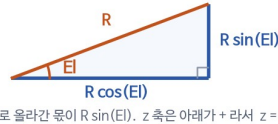
z축 (아래)

세 축 방향으로 간 거리 셋 (x, y, z) 이 직교좌표다.

(b) 공식은 어디서 나왔나 — 직각삼각형, 두 번

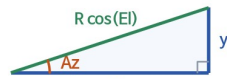
밑변 = 빗변 × cos, 높이는 빗변 × sin (삼각비의 정의)

① 고각 El로 위아래를 먼저 뚫는다



위로 올라간 길이 R sin(El). z 축은 아래가 +라서 z = -R sin(El)

② 남은 수평 성분을 Az로 다시 나눈다



$$x = R \cos(El) \cos(Az) \quad y = R \cos(El) \sin(Az)$$

직교좌표란

서로 직각인 세 축 위의 거리 세 개로 점 하나를 적는 방식.
극좌표와 같은 점, 표기만 다르다.

공식

$$\begin{aligned} x &= R \cos(El) \cos(Az) \\ y &= R \cos(El) \sin(Az) \\ z &= -R \sin(El) \end{aligned}$$

왜 직교좌표로 바꾸나

극좌표끼리는 회전을 합성할 수 없다.
회전행렬은 벡터에만 곱할 수 있다.

$$\text{예제값 } R = 20,000 \text{ m} \cdot Az = 30^\circ \cdot El = 0^\circ \rightarrow x = +17,320.5081 \text{ m} (= 10,000\sqrt{3}), \quad y = +10,000.0000 \text{ m}, \quad z = 0.0000 \text{ m}$$

역변환은 asin 이 아니라 atan2 — 반올림으로 인자가 1 을 살짝 넘으면 asin 은 NaN 을 내고, $\pm 90^\circ$ 근처에서 정밀도가 무너지기 때문입니다.

8

발표 대본

좌표계 · 약 80초

직교좌표가 무엇인지부터 말씀드리겠습니다. 서로 직각인 세 축을 잡아 놓고, 그 축 방향으로 각각 얼마씩 갔는지를 거리 세 개로 적는 방식입니다. 앞 장의 극좌표와 같은 점이고 담긴 정보도 같습니다. 거리 하나와 각도 둘로 적던 것을 거리 셋으로 바꿔 적는 것뿐입니다.

왜 바꾸느냐 하면, 극좌표끼리는 회전을 합성할 수 없기 때문입니다. 회전행렬은 벡터에만 곱할 수 있으니, 각도로 표현된 방향을 세 성분으로 풀어쓰는 준비 단계입니다.

공식은 세 줄입니다. x 는 $R \cos(El) \cos(Az)$ 로 보여사이트 방향 성분, y 는 $R \cos(El) \sin(Az)$ 로 오른쪽 성분, z 는 $-R \sin(El)$ 로 아래 방향 성분입니다.

이 공식이 어디서 나왔는지가 왼쪽 그림입니다. 외울 것이 아니라 직각삼각형의 삼각비를 두 번 쓴 것이 전부입니다. 삼각비는 한 줄입니다. 직각삼각형에서 밑변은 빗변 곱하기 코사인, 높이는 빗변 곱하기 사인입니다.

첫 번째는 고각 El로 위아래를 뚫습니다. 빗변이 R 이고 각이 El 인 직각삼각형에서, 밑변 즉 수평면에 누인 길이가 $R \cos(El)$ 이고, 높이는 즉 위로 올라간 길이가 $R \sin(El)$ 입니다. z 축은 아래가 양수라서 부호를 뒤집어 $z = -R \sin(El)$ 이 됩니다. 세 줄 중 z 에만 마이너스가 붙는 이유가 이것입니다.

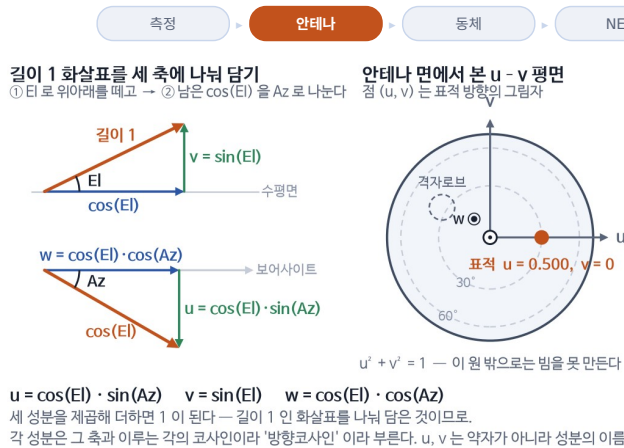
두 번째는 남은 수평 성분을 방위각 Az로 다시 나눕니다. 이번에는 빗변이 $R \cos(El)$ 이고 각이 Az 인 직각삼각형입니다. 밑변이 앞쪽 성분 x, 높이가 오른쪽 성분 y 가 됩니다. 그래서 x 는 $R \cos(El) \cos(Az)$, y 는 $R \cos(El) \sin(Az)$ 입니다.

정리하면 $\cos(El)$ 로 표적을 먼저 수평면에 눕히고, 그 길이를 Az로 앞과 오른쪽에 나눠 담는 것입니다. 예제값은 El이 0이라 눕힐 것이 없고, x 는 $+17,320.5081 \text{ m}$, 즉 $10,000 \sqrt{3}$ 이고, y 는 $+10,000 \text{ m}$, z 는 0 이 됩니다.

역변환에서는 asin 대신 atan2 를 씁니다. 반올림으로 인자가 1 을 살짝 넘으면 asin 은 NaN 을 내고, $\pm 90^\circ$ 근처에서 정밀도가 무너지기 때문입니다.

01. 안테나 좌표계 ③ UV (방향코사인)

표적 방향을 길이 1 인 화살표로 보고, 그 화살표를 안테나 세 축에 나눠 담은 값입니다



왜 UV 를 쓰나

현대 레이다는 대부분 위상배열이다.
 수백 ~ 수천 개 소자에 위상차를 줘서
 빔 방향을 전자적으로 바꾼다.

$$\psi = -k(m \cdot d \cdot u_0 + n \cdot d \cdot v_0)$$

소자 번호에 u, v 를 곱하기만 하면 된다.
 각도 (Az, EI) 로는 이렇게 단순해지지 않는다.
 되돌릴 때는 asin 대신 atan2 를 쓴다.
 $Az = \text{atan2}(u, w)$
 $EI = \text{atan2}(v, \sqrt{u^2 + w^2})$

빔 굵기가 조향각과 무관하게 일정하고, 격자로브 위치도 $u_0 \pm \lambda/d$ 로 단순하게 나옵니다.
 UV 에는 거리가 없습니다 — 방향만 담은 표현이라, 표적 위치를 옮기는 이번 변환 체인에는 들어가지 않습니다.

9

발표 대본

좌표계 · 약 60초

표적이 어느 쪽인지는 길이 1 짜리 화살표 하나로 적을 수 있습니다. 그 화살표를 안테나의 세 축에 나눠 담은 양이 u, v, w 입니다. 각 성분이 그 축과 이루는 각의 코사인이라서 방향코사인이라고 부릅니다. u 와 v 는 무언가의 약자가 아니라 그냥 성분의 이름입니다.

공식이 그렇게 생긴 이유는 두 번에 나눠 담기 때문입니다. 먼저 고각으로 위아래를 뺍니다. 위로 올라간 몫이 v, 즉 $\sin(EI)$ 이고 수평면에 누운 길이가 $\cos(EI)$ 입니다. 그 누운 길이를 방위각으로 다시 나누면 보어사이트 쪽이 $w = \cos(EI)\cos(Az)$, 오른쪽이 $u = \cos(EI)\sin(Az)$ 입니다. 셋을 제곱해 더하면 1 이 되는데, 길이 1 인 화살표를 나눠 담은 것이니 당연합니다.

쓰는 이유는 현대 레이다가 대부분 위상배열이기 때문입니다. 소자에 줘야 할 위상차가 소자 번호에 u, v 를 곱한 값에 정비례합니다. 각도로는 이렇게 단순해지지 않습니다. 빔 굵기가 조향각과 무관하게 일정한 것도, 격자로브 위치가 단순하게 나오는 것도 같은 이유입니다.

이번 과제에 안 들어가는 이유는 분명합니다. UV 에는 거리가 없습니다. 방향만 담은 표현이라 표적의 위치를 옮기는 변환 체인에는 쓸 수가 없습니다. UV 는 안테나가 빔을 어디로 쏠지 정하는 데 쓰는 표현입니다.

01. 동체 (Body) 좌표계

배에 실린 모든 장비의 공통 기준 — 안테나 · INS (Inertial Navigation System) · 무장이 여기로 모입니다



10

발표 대본

좌표계 · 약 60초

배에 실린 모든 장비의 공통 기준입니다. 안테나, INS, 무장이 여기로 모입니다.

원점은 배의 무게중심, 축은 x가 선수, y가 우현, z가 아래라서 FRD라고 부릅니다. 왼쪽 그림이 그 정의입니다. 세 축이 배에 붙어 배와 함께 움직입니다.

오른쪽 자세각 세 가지는 배 대신 레이더 안테나로 그렸습니다. 안테나가 동체에 볼트로 고정된 고정형이라 배가 기운 각이 그대로 안테나가 기운 각이고, 우리가 이 세 각을 쓰는 곳이 결국 안테나가 켜 값을 얼마나 되돌려야 하는가이기 때문입니다. roll은 x축 둘레로 뒤에서 본 모습, pitch는 y축 둘레로 옆에서 본 모습, yaw는 z축 둘레로 위에서 본 모습입니다.

주의할 점은 합치는 순서입니다. 회전은 곱셈 순서를 바꾸면 결과가 달라집니다. 그래서 yaw, pitch, roll 순서인 3-2-1을 문서로 고정해야 합니다. 구현에서도 $R_z(yaw) \cdot R_y(pitch) \cdot R_x(roll)$ 순서로 곱합니다.

한 가지 더 짚어 두면, 이 세 각이 들어가는 곳은 동체에서 NED로 가는 3단계입니다. 안테나에서 동체로 가는 2단계는 설치 방위와 레버암만 쓰고 자세각은 쓰지 않습니다. 고정형이라 그 값이 상수이기 때문입니다.

01. INS 좌표계

이번 설계에서는 동체 좌표계와 일치하므로, 좌표계가 아니라 값의 공급원으로만 쓰입니다

측정

안테나

동체

NED

ECEF

LLA

INS 는 무엇을 재고, 무엇을 주나

① 자이로 3 개 — 지금 얼마나 빨리 돌고 있나

초당 몇 도로 도는지만 쟀다. 그 값을 시간에 따라 계속 더하면 (적분) ' 처음보다 몇 도 돌아갔나 ' 가 되고, 그것이 곧 동체의 roll · pitch · yaw 다.

② 가속도계 3 개 — 지금 얼마나 빨라지고 있나

한 번 더하면 속도, 두 번 더하면 위치가 된다. 가만히 있어도 중력 1 g 가 같이 잡히므로 중력분을 먼저 빼고 더해야 한다.

③ 바깥은 전혀 보지 않는다 — 그래서 강하고, 그래서 흘러간다

위성도 지형도 안 본다. 전파방해에 강하고 빠르지만 (100 Hz 이상), 계속 더하는 방식이라 작은 오차도 시간이 지나면 쌓인다. 그래서 GPS 로 주기적으로 잡아 준다.

④ 스트랩다운 — 요즘 함정 INS 는 사실상 전부 이 방식

이번 과제만 그런 것이 아니라 업계 표준이다. 센서를 동체에 그대로 고정해 배와 함께 기울게 두고, 수평은 기계가 아니라 계산이 맞춘다. 예전 짐벌형은 모터가 센서를 늘 수평으로 받쳐 주는 방식이었는데, 정밀한 대신 크고 무겁고 비쌌다. 둘의 구조 비교는 부록 A 에 있습니다.

"INS 는 플랫폼 무게중심과 축이 일치한 상태로 설치되어 있음"

INS 좌표계 = 동체 (Body) 좌표계

설치 오차 0 · 레버암 0

따라서 INS 를 독립된 변환 단계로 두지 않습니다.
좌표계가 아니라 값의 공급원으로만 쓰입니다.

INS 가 주는 값	어디에 쓰이는가
동체의 roll · pitch · yaw	3 단계 회전행렬 C _{ned←body}
함선의 위도, 경도, 고도	4 단계 로컬→ECEF 회전과 평행이동
(함선의 속도)	표적 추적 필터 · 도플러 보정 — 이 발표 범위 밖

11

발표 대본

좌표계 · 약 80초

INS 는 관성항법장치입니다. 이름은 거창하지만 하는 일은 단순합니다. 자이로와 가속도계가 쟀 값을 계속 더해 나가는 장치입니다.

첫째, 자이로 세 개는 지금 얼마나 빨리 돌고 있는지만 쟀니다. 초당 몇 도로 도는가입니다. 그 값을 시간에 따라 계속 더하면, 즉 적분하면 처음보다 몇 도 돌아갔는지가 나옵니다. 그것이 곧 동체의 roll, pitch, yaw 입니다. 속도계만 보고 주행거리를 아는 것과 같은 원리입니다.

둘째, 가속도계 세 개는 지금 얼마나 빨라지고 있는지만 쟀니다. 한 번 더하면 속도, 두 번 더하면 위치가 됩니다. 다만 가만히 서 있어도 중력 1 g 가 같이 잡히기 때문에 중력분을 먼저 빼고 더해야 합니다.

셋째, INS 는 바깥을 전혀 보지 않습니다. 위성도 지형도 보지 않고 자기 안의 센서만 씁니다. 그래서 전파방해에 강하고 갱신이 빠릅니다. 100 Hz 이상 나옵니다. 대신 계속 더하는 방식이라 작은 오차도 시간이 지나면 쌓입니다. 그래서 GPS 같은 외부 기준으로 주기적으로 잡아 줍니다.

넷째, 요즘 함정 INS 는 사실상 전부 스트랩다운 방식입니다. 이번 과제에서만 그런 것이 아니라 업계 표준입니다. 센서를 동체에 그대로 볼트로 고정해 배와 함께 기울게 두고, 수평은 기계가 아니라 계산이 맞춥니다. 예전에 쓰던 짐벌형은 모터가 센서를 늘 수평으로 받쳐 주는 방식이었는데, 정밀한 대신 크고 무겁고 비쌌습니다. 둘의 구조 비교는 부록 A 에 있습니다.

이번 설계에서 중요한 것은 조건 하나입니다. INS 는 플랫폼 무게중심과 축이 일치한 상태로 설치되어 있다. 그러면 INS 좌표계가 곧 동체 좌표계이고, 설치 오차도 레버암도 0 입니다. 따라서 INS 를 독립된 변환 단계로 두지 않습니다.

INS 는 좌표계가 아니라 값의 공급원입니다. 오른쪽 표가 그 목록입니다. 동체의 roll, pitch, yaw 는 3단계 회전행렬에 들어가고, 함선의 위도 · 경도 · 고도는 4단계 로컬에서 ECEF 로 가는 회전과 평행이동에 들어갑니다. 속도도 주지만 추적 필터와 도플러 보정에 쓰는 값이라 이 발표 범위 밖입니다.

01. NED / ENU 국지수평 좌표계

배가 아무리 흔들려도 북쪽은 계속 북쪽입니다

측정

안테나

동체

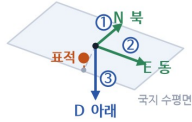
NED

ECEF

LLA

NED (항공우주 · 항법 · 무기체계)

North-East-Down. 북 → 동 → 아래 순서, 셋째 축이 아래로 +

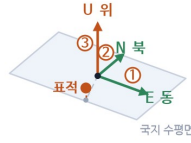


(N -19,340, E +5,155, D -10) m

표적 높이는 부풀린 그림

ENU (측지 · 측량 · GIS · 로봇틱스)

East-North-Up. 동 → 북 → 위 순서, 셋째 축이 위로 +



(E +5,155, N -19,340, U +10) m

북 · 동은 같은 방향이다. 셋째 축만 뒤집히고, 둘 다 오른손 좌표계다.

이 좌표계의 존재 이유

배와 함께 이동하지만
함께 회전하지는 않는다.그래서 표적이 실제로 움직인 것과
배가 흔들린 것을 구분할 수 있다.

추적 필터를 여기서 돌리는 이유다.

"아래"는 지구 중심 방향이 아니다

타원체 법선의 반대 방향이다. 지구 중심 방향과는 위도 45° 부근에서 최대 약 0.19° 어긋나는데, 이 둘을 혼동하면 지표 위치가 약 21 km 틀어진다.

12

발표 대본

좌표계 · 약 45초

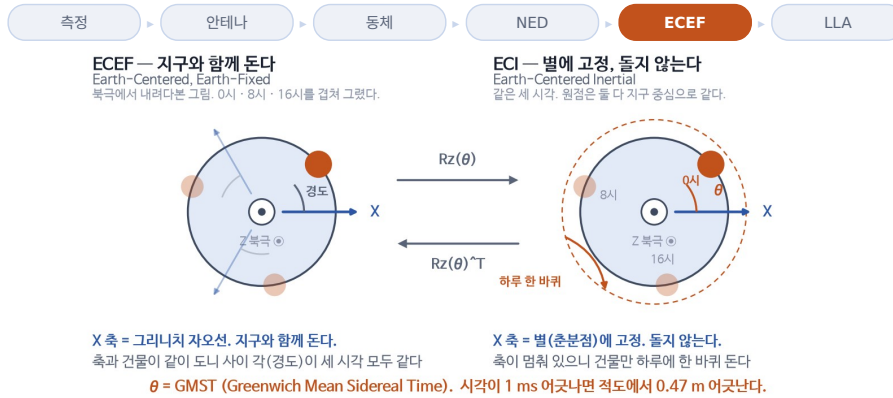
그림 두 개는 일부러 똑같이 그렸습니다. NED와 ENU는 북과 동이 가리키는 방향이 같고, 축을 세는 순서와 셋째 축의 방향만 다릅니다. 둘 다 오른손 좌표계이고, 이번 과제는 NED를 씁니다.

배가 아무리 흔들려도 북쪽은 계속 북쪽입니다. 이 좌표계의 존재 이유가 그것입니다. 배와 함께 이동하지만 함께 회전하지는 않습니다. 그래서 표적이 실제로 움직인 것과 배가 흔들린 것을 구분할 수 있고, 추적 필터를 여기서 돌립니다.

한 가지 주의점이 있습니다. "아래"는 지구 중심 방향이 아니라 타원체 법선의 반대 방향입니다. 지구 중심 방향과는 위도 45도 부근에서 최대 약 0.19도 어긋나는데, 이 둘을 혼동하면 지표 위치가 약 21 km 틀어집니다.

01. ECEF / ECI 지구 중심 좌표계

원점은 같고, 축이 도느냐 마느냐만 다릅니다



ECEF 는 왜 필요한가

로컬 좌표계에서 위경도로 곧장 갈 수 없다.
로컬 원점 (배 위치) 이 지구 어디인지는
지구 기준 좌표로만 표현되기 때문이다. → 다리 역할

ECI 는 이번 설계에 필요 없다

뉴턴 법칙은 회전하지 않는 좌표계에서만 그대로 성립.
그래서 중력만 받고 날아가는 물체 (위성 · 탄도탄) 는 ECI 에서 계산.
항공기 · 수상함 표적은 ECEF/NED 로 충분하다.

13

발표 대본

좌표계 · 약 40초

둘 다 원점은 지구 중심이고, 축이 지구와 함께 도느냐 마느냐만 다릅니다.

ECEF 가 필요한 이유는 로컬 좌표계에서 위경도로 곧장 갈 수 없기 때문입니다. 로컬 원점, 즉 배의 위치가 지구 어디인지는 지구 기준 좌표로만 표현됩니다. 다리 역할입니다.

ECI 는 이번 설계에 필요 없습니다. 뉴턴 법칙은 회전하지 않는 좌표계에서만 그대로 성립하므로, 중력만 받고 날아가는 위성이나 탄도탄은 ECI 에서 계산합니다. 항공기나 수상함 표적은 ECEF 와 NED 로 충분합니다.

01. LLA 측지 좌표계

우리가 아는 그 지도 좌표 — 다만 지구는 구가 아닙니다

측정

안테나

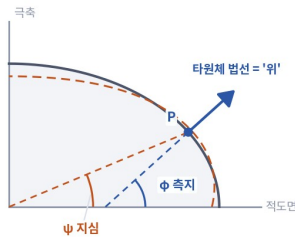
동체

NED

ECEF

LLA

지구 타원체 · 측지위도 · 고도 기준 (편평률을 크게 과장한 그림)



두 위도의 차이

$$\tan \psi = (1 - e^2) \cdot \tan \phi, \text{ 최대 } 0.192^\circ$$

→ 지표에서 약 21 km 에 해당

고도의 두 기준

— 타원체 (WGS-84) — LLA 의 고도 기준

— 지오이드 — 해도·고도계의 "해발" 기준

해발고도 = 타원체고 - 지오이드고

한국 근해 지오이드고 $\approx +20 \sim +25$ m시스템 내부는 타원체고로 통일하고,
표시할 때만 변환하는 것이 안전하다.

WGS-84 타원체

a (장반경) 6,378,137.0 m

b (단반경) 6,356,752.3 m

1/f 298.257223563

적도 반지름과 극 반지름의 차이가
약 21 km. 지구 크기에 비하면 0.3 %
정도지만, 구로 계산하면 위치가
20 km 넘게 어긋난다 (위도 45° 부근).

World Geodetic System 1984 · GPS · 해도
기준

고도의 함정 : LLA 의 고도는 타원체고 (HAE) — 해도·고도계의 "해발" 과 다릅니다

HAE = Height Above Ellipsoid. 해발고도 = 타원체고 - 지오이드고. 한국 근해 지오이드고는 약 +20 ~ +25 m. 시스템 내부는 하나로 통일할 것.

14

발표 대본

좌표계 · 약 50초

우리가 아는 그 지도 좌표입니다. 다만 지구는 구가 아닙니다.

WGS-84 타원체는 장반경 6,378,137.0 m, 단반경 6,356,752.3 m, 편평률의 역수 298.257223563 입니다.
적도 반지름과 극 반지름의 차이가 약 21 km 로, 지구 크기에 비하면 0.3 % 정도지만 구로 계산하면 위도 45도 부
근에서 위치가 20 km 넘게 어긋납니다. GPS 와 해도가 쓰는 것도 이것입니다.

고도에 함정이 하나 있습니다. LLA 의 고도는 타원체고, HAE 이고 해도나 고도계의 "해발" 과 다릅니다. 해발고도
는 타원체고에서 지오이드고를 뺀 값이고, 한국 근해 지오이드고는 약 +20 에서 +25 m 입니다. 시스템 내부는 하
나로 통일해야 합니다.

01. 변환 체인

회전은 설치 방위와 INS(자세 · 위경도) 가 , 이동은 레버암과 INS(배 위치) 가 공급합니다 . 이 구분이 그대로 오차의 출처 목록이 됩니다 .

신호처리 출력

R, Az, El (안테나 극좌표)

안테나 좌표계

Polar → Cartesian → (UV)

동체 (Body) 좌표계

x= 선수 y= 우현 z= 아래 (FRD)

로컬 (NED) 좌표계

x= 북 y= 동 z= 아래 ★ 데이터 처리

ECEF 좌표계

지구중심 지구고정 직교좌표

LLA 좌표계

위도 · 경도 · 고도 → 전시기

극좌표를 x, y, z 세 성분으로 분해

표현만 바꿀 뿐 · 회전도 이동도 없음

$C_{body \leftarrow ant} = R_z(\text{설치방위}) \cdot R_y(\text{백틸트}) + \text{레버암}$

회전 + 이동 설치 도면이 공급 · 고정형이라 상수

$C_{ned \leftarrow body} = R_z(yaw) \cdot R_y(pitch) \cdot R_x(roll)$

회전 INS 자세가 공급 · 매 순간 변환

$C_{ecef \leftarrow ned}(\text{위도}, \text{경도}) + \text{배의 ECEF 위치}$

회전 + 이동 INS 위치가 공급

위도 · 고도 반복 계산 (값이 안 변할 때까지)

WGS-84 타원체 · 직교좌표 → 위경도

표기 : $C_{ned \leftarrow body}$ 는 동체 좌표를 NED 좌표로 바꾸는 회전행렬 .
 $v_{ned} = C_{ned \leftarrow body} \times v_{body}$ — 화살표 방향이 곧 변환 방향 .
 거꾸로 갈 때는 전치 $C_{ned \leftarrow body}^T$ 를 곱한다 .

15

발표 대본

설계 · 약 60초

전체 체인을 한 장으로 보겠습니다. 왼쪽이 좌표계 순서, 오른쪽이 각 단계의 변환입니다.

신호처리 출력 R, Az, El 이 안테나 극좌표입니다. 1단계에서 이것을 x, y, z 세 성분으로 분해합니다. 표현만 바꿀 뿐 회전도 이동도 없습니다.

2단계 안테나에서 동체로는 $R_z(\text{설치방위})$ 곱하기 $R_y(\text{백틸트})$ 로 회전하고 레버암을 더합니다. 설치 도면이 공급하고, 고정형이라 상수입니다.

3단계 동체에서 NED 로는 $R_z(yaw) \cdot R_y(pitch) \cdot R_x(roll)$ 로 회전만 합니다. INS 자세가 공급하고 매 순간 변환합니다. 여기가 데이터 처리 구간입니다.

4단계 NED 에서 ECEF 로는 위도 · 경도로 정해지는 회전에 배의 ECEF 위치를 더합니다. INS 위치가 공급합니다.

5단계 ECEF 에서 LLA 로는 위도와 고도를 번갈아 계산하는 반복입니다. WGS-84 타원체 기준입니다.

회전은 설치 방위와 INS 의 자세 · 위경도가, 이동은 레버암과 INS 의 배 위치가 공급합니다. 이 구분이 그대로 오차의 출처 목록이 됩니다. 표기를 하나 짚고 가면, $C_{ned \leftarrow body}$ 는 동체 좌표를 NED 좌표로 바꾸는 회전행렬이고 화살표 방향이 곧 변환 방향입니다. 거꾸로 갈 때는 전치를 곱합니다.

02

실전 설계

- 1 설계 조건
- 2 단계별 좌표계 선정
- 3 단계별 변환 ① ~ ③ — 안테나 → 동체 → 로컬
- 4 단계별 변환 ④ ~ ⑤ — 로컬 → ECEF → 위경도
- 5 역변환

16

발표 대본

장절 · 약 30초

Part 2 는 실전 설계입니다. 함선 고정형 안테나라는 조건을 놓고, 주어진 조건 여섯 가지가 설계를 어떻게 묶는지, 다섯 단계에 어떤 좌표계를 쓰고 무엇을 쓰지 않는지를 정합니다. 그리고 예제값 하나를 안테나에서 위경도까지 실제로 따라가고, 마지막에 되돌아오는 역변환까지 봅니다.

02. 설계 조건

주어진 조건 여섯 가지와, 그것이 설계를 어떻게 묶는가

조건	내용	설계에 미치는 영향
① 플랫폼	해상 함선, 고정형 안테나	안테나가 돌지 않는다 → 회전행렬이 상수 . 대신 배의 흔들림을 전부 계산으로 보상
② 안테나 설치	선수 기준 시계방향 90°, 무게중심에서 30 m 뒤 / 10 m 위	설치 방위 90°, 백틸트 0°, 레버암 (-30, 0, -10) m
③ INS 설치	무게중심과 축 일치	INS 좌표계 = 동체 좌표계 → 변환 단계 하나가 통째로 생략
④ 입력	안테나 기준 거리 · 방위각 · 고각	체인의 출발점이 안테나 극좌표로 확정
⑤ 처리	로컬 좌표계에서 수행	체인의 중간 목적지가 NED 로 확정
⑥ 출력	위 / 경 / 고도 + 안테나 극좌표	정변환뿐 아니라 역변환도 필요

④·⑤·⑥ 이 체인의 출발점 · 중간점 · 도착점을 이미 지정합니다 . 설계자가 정할 것은 그 사이를 어떻게 이을 것인가입니다 .

17

발표 대본

설계 · 약 60초

과제 조건 여섯 가지와 그것이 설계를 어떻게 묶는지입니다.

조건 1, 해상 함선에 고정형 안테나입니다. 안테나가 돌지 않으니 회전행렬이 상수이고, 대신 배의 흔들림을 전부 계산으로 보상해야 합니다.

조건 2, 안테나는 선수 기준 시계방향 90도, 무게중심에서 30 m 뒤, 10 m 위입니다. 설치 방위 90도, 백틸트 0도, 레버암 (-30, 0, -10) m 가 됩니다.

조건 3, INS 는 무게중심과 축이 일치합니다. INS 좌표계가 동체 좌표계와 같아져서 변환 단계 하나가 통째로 생략 됩니다.

조건 4, 입력은 안테나 기준 거리 · 방위각 · 고각입니다. 체인의 출발점이 안테나 극좌표로 확정됩니다.

조건 5, 처리는 로컬 좌표계에서 합니다. 체인의 중간 목적지가 NED 로 확정됩니다.

조건 6, 출력은 위 · 경 · 고도와 안테나 극좌표 둘 다입니다. 정변환뿐 아니라 역변환도 필요합니다.

조건 4, 5, 6 이 출발점, 중간점, 도착점을 이미 지정합니다. 설계자가 정할 것은 그 사이를 어떻게 이을 것인가입니다.

02. 단계별 좌표계 선정

단계	좌표계	왜 이것인가
0	안테나 — 극좌표	레이다가 물리적으로 잴 수 있는 것이 거리와 두 각도뿐
1	안테나 — 직교좌표	회전행렬은 벡터에만 곱할 수 있다
2	동체 (Body)	안테나 · INS · 무장이 만나는 유일한 공통 기준 . 배를 거치지 않고 바깥으로 나갈 방법이 없다
3	로컬 (NED)	배가 흔들려도 이 좌표계는 흔들리지 않는다 . ENU 가 아닌 NED 인 이유는 동체 FRD 와 축 의미가 맞아서
4	ECEF	로컬에서 위경도로 공장 갈 수 없다 — 다리 역할
5	LLA	전시기 · 통제제어가 지도 위에 표시하는 형식

쓰지 않은 좌표계

INS

③ 에 의해 동체와 일치 . 항등행렬을
곱하는 셈

UV

빔 조향은 안테나 내부의 일 — 데이터 처리
체인 밖

ECI

관성 동역학 전파가 필요할 때만 (위성 ·
탄도탄)

ENU

NED 와 정보량이 같다 — 하나로 통일

18

발표 대본

설계 · 약 60초

단계별로 어떤 좌표계를 골랐고 왜 골랐는지입니다.

0단계 안테나 극좌표는 레이다가 물리적으로 잴 수 있는 것이 거리와 두 각도뿐이기 때문입니다. 1단계 안테나 직교좌표는 회전행렬이 벡터에만 곱해지기 때문입니다.

2단계 동체는 안테나 · INS · 무장이 만나는 유일한 공통 기준이고, 배를 거치지 않고 바깥으로 나갈 방법이 없기 때문입니다.

3단계 로컬 NED 는 배가 흔들려도 흔들리지 않기 때문입니다. ENU 가 아닌 NED 인 이유는 동체 FRD 와 축 의미가 맞아서입니다.

4단계 ECEF 는 로컬에서 위경도로 공장 갈 수 없어서 다리 역할이고, 5단계 LLA 는 전시기와 통제제어가 지도에 표시하는 형식입니다.

쓰지 않은 좌표계도 이유가 있습니다. INS 는 조건 3 에 의해 동체와 일치해서 항등행렬을 곱하는 셈이고, UV 는 빔 조향이라 데이터 처리 체인 밖이고, ECI 는 관성 동역학 전파가 필요할 때만 쓰고, ENU 는 NED 와 정보량이 같아서 하나로 통일했습니다.

02. 단계별 변환 ① ~ ③ 안테나 → 동체 → 로컬

1 안테나 극좌표 → 직교좌표

표현 변경 (회전·이동 없음)

$$\begin{aligned} x &= R \cdot \cos(El) \cdot \cos(Az) = +17,320.5081 \quad (= 10,000\sqrt{3}) \\ y &= R \cdot \cos(El) \cdot \sin(Az) = +10,000.0000 \\ z &= -R \cdot \sin(El) = 0.0000 \end{aligned}$$

2 안테나 → 동체

회전 + 평행이동

$$\begin{aligned} C_{\text{body} \leftarrow \text{ant}} &= R_z(90^\circ) \cdot R_y(0^\circ) \\ p_{\text{body}} &= C \times p_{\text{ant}} + (-30, 0, -10) \\ &= (-10,030.0, +17,320.5, -10.0) \end{aligned}$$

3 동체 → 로컬 (NED)

회전만 (원점이 같음)

$$\begin{aligned} C_{\text{ned} \leftarrow \text{body}} &= R_z(45^\circ) \cdot R_y(0^\circ) \cdot R_x(0^\circ) \\ p_{\text{ned}} &= (N -19,339.73, E +5,155.17, D -10.0) \\ \text{진북 기준 방위 } &165.074^\circ \\ \text{검산 : } &45^\circ + 90^\circ + 30^\circ = 165^\circ. \text{ 배가 수평이라 각도가 그냥 더해진다. } 0.074^\circ \text{ 차이는 레버암 } 30 \text{ m} \text{ 때문} \end{aligned}$$

여기가 데이터 처리 구간 — 추적·필터링은 로컬 좌표계에서 수행합니다

19

발표 대본

설계 · 약 65초

예제값으로 앞 세 단계를 따라가 보겠습니다.

1단계, 극좌표를 직교좌표로. 표현만 바꿉니다. $x = +17,320.5081$, 즉 $10,000$ 루트 3 이고, $y = +10,000$, $z = 0$ 입니다.

2단계, 안테나에서 동체로. 회전과 평행이동입니다. $C_{\text{body} \leftarrow \text{ant}}$ 는 $R_z(90^\circ) \cdot R_y(0^\circ)$ 이고, 여기에 곱한 뒤 $(-30, 0, -10)$ 을 더하면 $(-10,030.0, +17,320.5, -10.0)$ 입니다. x 와 y 가 자리를 바꾸고 부호가 뒤집힌 것이 보입니다.

3단계, 동체에서 NED 로. 원점이 같으니 회전만입니다. $C_{\text{ned} \leftarrow \text{body}}$ 는 $R_z(45^\circ) \cdot R_y(0^\circ) \cdot R_x(0^\circ)$ 이고, 결과는 $N -19,339.73$, $E +5,155.17$, $D -10.0$ 입니다. 진북 기준 방위 165.074° 도입니다.

검산도 됩니다. 배가 수평이라 각도가 그냥 더해져서 45 더하기 90 더하기 30 , 165° 도입니다. 계산값 165.074° 와의 0.074° 차이는 레버암 30 m 때문입니다. roll 이나 pitch 가 조금이라도 있으면 이 덧셈은 깨지는데, 그 이야기는 확인 실험에서 다시 보겠습니다.

여기가 데이터 처리 구간입니다. 추적과 필터링은 이 로컬 좌표계에서 수행합니다.

02. 단계별 변환 ④ ~ ⑤ 로컬 → ECEF → 위경도

4 로컬 → ECEF

회전 + 평행이동

- ① 배의 위치를 ECEF 로
- ② 로컬 벡터를 지구 방향으로 회전
(회전량은 배의 위도 · 경도가 결정)
- ③ 더한다

5 ECEF → LLA

위도 · 고도 반복 계산

담 - 달걀 관계라 딱 떨어지는 공식이 없다.
 위도를 알아야 곡률반경 N 을 구하는데
 N 을 알아야 위도를 구할 수 있기 때문.
 → 구로 본 위도에서 출발, 값이 안 변할 때까지 반복

표적 LLA = 36.233700252°N 127.364344961°E 고도 41.4952 m

왜 고도가 10 m 가 아니라 41.5 m 인가?

레이다는 수평 ($EI = 0^\circ$) 으로 쏘고 안테나는 10 m 높이니 "표적 고도 10 m" 일 것 같습니다.
 그런데 빔은 직진하는데 바다 표면은 멀어질수록 아래로 휘어 내려갑니다. 20 km 지점에서 그 차이가 31.5 m, $10 + 31.5 = 41.5$ m.

20

발표 대본

설계 · 약 70초

4단계, 로컬에서 ECEF 로. 회전과 평행이동입니다. 배의 위치를 ECEF 로 바꾸고, 로컬 벡터를 지구 방향으로 회전하고, 둘을 더합니다. 회전량은 배의 위도 · 경도가 결정합니다.

5단계, ECEF 에서 LLA 로. 위도와 고도가 서로 물려 있어서 딱 떨어지는 공식이 없습니다. 위도를 알아야 곡률반경 N 을 구하는데 N 을 알아야 위도를 구할 수 있기 때문입니다. 그래서 지구를 구로 본 위도에서 출발해서 값이 안 변할 때까지 반복합니다. 보통 5 ~ 7회면 수렴합니다.

표적 LLA 는 36.233700252°N, 127.364344961°E, 고도 41.4952 m 입니다.

여기서 질문이 하나 나옵니다. 왜 고도가 10 m 가 아니라 41.5 m 인가. 레이다는 수평, EI 0도로 쏘고 안테나는 10 m 높이니 표적 고도 10 m 일 것 같습니다. 그런데 빔은 직진하는데 바다 표면은 멀어질수록 아래로 휘어 내려갑니다. 20 km 지점에서 그 차이가 31.5 m 이고, 10 더하기 31.5 가 41.5 m 입니다. 앞에서 본 로컬 평면의 한계가 바로 이 숫자입니다.

02. 역변환

조건 ⑥ 은 결과를 두 가지 형태로 요구합니다

역변환은 새로 배울 것이 없습니다

① 회전은 전치 (transpose) ② 평행이동은 빼기 ③ 순서는 거꾸로

```

표적 LLA (36.233700°N, 127.364345°E, 41.4952 m)
→ ECEF          // 정변환과 같은 공식
→ 로컬 NED      C_ned←e × ( 표적 - 배 )
→ 동체          C_ned←body^T × p_ned      // 전치
→ 안테나        C_body←ant^T × (p_body - 레버암) // 전치 + 빼기
→ 극좌표        R = √(x²+y²+z²), Az = atan2(y, x), El = atan2(-z, √(x²+y²))

```

결과 : R = 20,000.000000 m Az = 30.000000° El = 0.000000°

입력값과 정확히 같은 값이 돌아왔습니다. 왕복오차 $\Delta R = 0.0$ m, $\Delta Az = 1.8e-12^\circ$ — 컴퓨터 반올림 수준입니다.

부호 하나, 전치 방향 하나만 틀려도 이 왕복 시험은 즉시 실패합니다 — 그래서 가장 강력한 검증 수단이 됩니다.

21

발표 대본

설계 · 약 60초

조건 6 은 결과를 두 가지 형태로 요구합니다. 그래서 역변환이 필요한데, 새로 배울 것이 없습니다. 회전은 전치, 평행이동은 빼기, 순서는 거꾸로입니다.

표적 LLA 에서 출발해 ECEF 로 가는 것은 정변환과 같은 공식이고, 로컬 NED 는 표적에서 배를 뺀 것을 회전하고, 동체는 $C_{ned} \leftarrow body^T \times p_{ned}$ 의 전치를 곱하고, 안테나는 레버암을 뺀 뒤 $C_{body} \leftarrow ant^T$ 의 전치를 곱하고, 마지막에 극좌표로 돌아옵니다. R 은 제곱합의 제곱근, Az 는 $atan2(y, x)$, El 은 $atan2(-z, \sqrt{x^2+y^2})$ 입니다.

결과는 $R = 20,000.000000$ m, $Az = 30.000000^\circ$, $El = 0.000000^\circ$ 입니다. 입력값과 정확히 같은 값이 돌아왔습니다. 왕복오차는 ΔR 0.0 m, ΔAz $1.8e-12^\circ$ 로 컴퓨터 반올림 수준입니다.

부호 하나, 전치 방향 하나만 틀려도 이 왕복 시험은 즉시 실패합니다. 그래서 가장 강력한 검증 수단이 됩니다.

03

C 로 확인

1 구현 원칙

2 결과 확인

3 확인 실험

22

발표 대본

장절 · 약 25초

Part 3 은 C 로 짜서 확인하는 이야기입니다. 좌표변환은 적당히 그럴듯한 값이 나오기 때문에 눈으로는 검증되지 않습니다. 그래서 애초에 실수하기 어렵게 만드는 구현 원칙, 콘솔과 UI 로 보는 결과와 왕복 오차, 그리고 설계 판단이 실제로 근거가 있는지 보는 확인 실험, 이렇게 세 가지를 다룹니다.

03. 구현 원칙

좌표변환 버그의 특징은 컴파일도 되고 값도 그럴듯해 보인다는 것입니다

① 함수 이름에 방향을 박는다

```
f_BodyToNed() / f_NedToBody()
f_AntToBody() / f_BodyToAnt() / f_EcefToLla()
```

② 회전은 Rx·Ry·Rz 곱으로만 만든다

```
Matrix_Product3(&Rz, &Ry, &Rx) / 역변환은 Matrix_Transpose()
행렬 연산은 참고 소스 matrixCalcLib 그대로. 원소를 손으로 안 쓴다
```

③ 단위는 안에서 하나로 통일한다

```
rad · m 만 쓴다. deg 는 DEG2RAD() / RAD2DEG() 로 입출력에서만
중간에 deg 가 섞이면 값이 그럴듯해서 더 못 잡는다
```

검증 — 갔다가 돌아오면 원래 값이 나와야 합니다

왕복 시험	측정값 → LLA → 측정값. 콘솔과 UI 가 입력과의 차이를 같이 찍음	부호 · 전치 방향 · 순서 실수
예제값 손계산	동체 좌표는 Rz(90°) 로 돌리고 오프셋을 더하면 손으로 나옴	레버암 부호, 설치 방위
자세 바꿔 가며	UI 슬라이더로 roll · pitch · yaw 를 다르게 놓고 왕복 확인	대칭 자세에선 안 드러나는 전치 오류

23

발표 대본

구현 · 약 80초

좌표변환 버그의 특징은 컴파일도 되고 값도 그럴듯해 보인다는 것입니다. 그래서 애초에 실수하기 어렵게 세 가지 원칙을 두었습니다.

첫째, 함수 이름에 방향을 박았습니다. f_BodyToNed 와 f_NedToBody, f_AntToBody 와 f_BodyToAnt 처럼 어디서 어디로 가는지가 이름에 있습니다. 인자 순서를 헷갈릴 자리가 없습니다.

둘째, 회전은 Rx, Ry, Rz 곱으로만 만들었습니다. 행렬 연산은 참고 소스로 받은 matrixCalcLib 을 그대로 썼습니다. 위치는 3x1, 회전은 3x3 matrix 로 두고, 동체에서 NED 로 가는 회전은 Matrix_Product3 에 Rz, Ry, Rx 를 넣어 만들고, 역변환은 Matrix_Transpose 하나입니다. 행렬 원소를 손으로 쓰지 않으니 부호 실수가 끼어들 자리가 없습니다.

셋째, 단위는 안에서 하나로 통일했습니다. 라이브러리 안에서는 라디안과 미터만 쓰고, 도는 DEG2RAD, RAD2DEG 로 입출력에서만 바꿉니다. 중간에 도가 섞이면 값이 그럴듯해서 더 못 잡습니다.

검증 원리는 하나입니다. 갔다가 돌아오면 원래 값이 나와야 합니다. 콘솔과 UI 가 정변환 뒤 역변환한 값을 입력과 비교해서 차이를 같이 찍고, 예제값의 동체 좌표는 손으로 계산한 값과 맞춰 봤습니다. 전치 방향이 뒤바뀐 실수는 대칭적인 자세에서는 드러나지 않기 때문에, UI 슬라이더로 roll, pitch, yaw 를 전부 다른 값으로 놓고 왕복오차가 그대로 반올림 수준인지 확인합니다.

03. 결과 확인

같은 CoordCore 를 들고 입력을 바꾸면 바로 다시 계산합니다 . 역변환이 입력값과 반올림 수준까지 일치하면 체인 전체가 맞은 것입니다

프로그램 구성

CoordCore

좌표변환 정적 라이브러리 (C)
coord_frames + 참고 소스 matrixCalcLib

radar_coord

콘솔 . 과제 3.1 입력값으로 단계별 값 출력
왕복오차까지 같이 찍음

CoordUI

MFC (Microsoft Foundation Classes) 다이얼로그
변환 로직 없이 f_* 함수만 부름

실행 프로젝트 둘 다 CoordCore 를
ProjectReference 로 물고 있음

CoordFrames

측정값 R [m] / Az / El [deg]

20000.0 30.000 0.000

플랫폼 위치 lat / lon [deg] / alt [m]

36.408 127.307 0.0

플랫폼 자세 [deg] roll / pitch / yaw

roll 0.0
pitch 0.0
yaw 45.0

안테나 설치 az / tilt [deg] / offset [m]

90.000 0.000 -30, 0, -10

예제값 복원

입력 각도 : 도 / 변환 계산 : 라디안, m

단계별 결과

단계	x / N / X	y / E / Y	z / D / Z
1) 극좌표 -> 안테나 직교	17320.5	10000.0	0.0
2) 안테나 -> 동체	-10030.0	17320.5	-10.0
3) 동체 -> NED	-19339.7	5155.2	-10.0
4) NED -> ECEF	-3125892.5	4093775.0	3749164.2
5) ECEF -> LLA	36.233700	127.364345	41.4952
역변환 -> 극좌표	20000.0	30.000000	0.000000

최종 출력

위 / 경 / 고도 36.233700252 N 127.364344961 E 41.4952 m
안테나 극좌표 R 20000.0000 m Az 30.000000° El 0.000000°
왕복오차 dR 0.0 m dAz 1.8e-12° dEl 1.7e-12°

1e-12 ° 왕복 각도 오차

0 건 컴파일 경고

< 1e-9 m 왕복 거리 오차

roll 슬라이더를 돌리면 다음 장 실험 B 의 고각 변화가 그 자리에서 보입니다 . 발표 중에 직접 돌려 봅니다 .

24

발표 대본

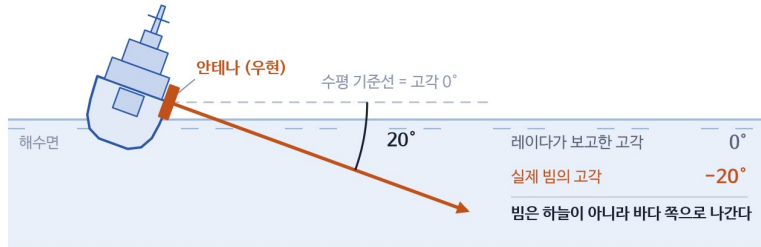
결과 · 약 75초

결과를 프로그램으로 확인합니다. 구성은 셋입니다. CoordCore 는 좌표변환을 담은 정적 라이브러리로 참고 소스 matrixCalcLib 을 그대로 씁니다. radar_coord 는 콘솔 프로그램으로 과제 3.1 입력값의 단계별 값과 왕복오차를 찍습니다. CoordUI 는 MFC 다이얼로그인데 변환 로직 없이 CoordCore 의 f_* 함수만 부릅니다.

화면 오른쪽이 CoordUI 입니다. 측정값, 플랫폼 위치와 자세, 안테나 설치값을 넣으면 바로 다시 계산해서 표에 단계별 값을, 아래에 최종 출력을 보여 줍니다. 최종 출력은 조건 6 이 요구한 두 형태입니다. 위경도와 고도는 36.23 37도, 127.3643도, 41.5 m 이고, 안테나 극좌표는 20 km, 30도, 0도로 입력값과 같습니다.

아래 세 숫자가 핵심입니다. 역변환한 값이 입력값과 각도로 1e-12도, 거리로 1e-9 m 안에서 일치하고, 컴파일 경고는 0건입니다. 컴퓨터 반올림 수준까지 맞다는 것은 변환 체인 전체가 서로 정확히 역관계라는 뜻입니다. 자세는 슬라이더로도 바꿀 수 있어서, roll 을 10도, 20도로 돌리면 다음 장 실험 B 의 고각 변화가 그 자리에서 보입니다. 발표 중에 직접 돌려 보겠습니다.

03. 확인 실험



실험 B 배가 기울면

같은 측정값 (Az 30°, El 0°) 에
자세만 바꿔 놓은 결과

roll	실제 고각
0°	+0.03°
10°	-8.61°
20°	-17.19°

레이다는 여전히 "고각 0도" 라고 보고합니다.
그건 안테나 기준일 뿐입니다.
roll 20° 면 20 km 앞 고도가 5.9 km 어긋난다.

실험 A 레버암을 빼먹으면

수평으로 정확히 30 m, 고도로 10 m 어긋난다.
거리가 멀어져도 이 오차는 줄지 않는다.

실험 C 지구 곡률

5 km 2 m · 20 km 31 m · 100 km 786 m.
거리의 제곱에 비례해 커진다.

각도를 더하는 방식으로는 이 효과를 만들 수 없습니다 — 해상 플랫폼에서 회전행렬은 선택이 아니라 필수입니다.

25

발표 대본

실험 · 약 60초

설계 판단이 실제로 근거가 있는지 세 가지 실험으로 확인했습니다.

실험 A, 레버암을 빼먹으면 수평으로 정확히 30 m, 고도로 10 m 어긋납니다. 거리가 멀어져도 이 오차는 줄지 않습니다.

실험 B, 배가 기울면. 같은 측정값 Az 30도, El 0도에 자세만 바꿔 놓았습니다. roll 0도에서 실제 고각 +0.03도, 10도에서 -8.61도, 20도에서 -17.19도입니다. 레이다는 여전히 "고각 0도" 라고 보고하지만 그건 안테나 기준일 뿐이고, roll 20도면 20 km 앞에서 고도가 약 5.9 km 어긋납니다.

실험 C, 지구 곡률. 5 km 에서 2 m, 20 km 에서 31 m, 100 km 에서 786 m 로 거리의 제곱에 비례해 커집니다.

각도를 더하는 방식으로는 이 효과를 만들 수 없습니다. 해상 플랫폼에서 회전행렬은 선택이 아니라 필수입니다.

정리

- 1 좌표계는 " 어디에 서서 어느 방향을 기준으로 재는가 " 이고 , 좌표변환은 그 기준을 맞추는 계산이다
- 2 변환의 재료는 회전과 평행이동 둘뿐이다
- 3 순서는 안테나 → 동체 → 로컬 → ECEF → 위경도 . 건너뛸 수 없다
- 4 회전은 설치 방위와 INS 자세 · 위경도가 , 평행이동은 레버암과 배의 위치가 공급한다
- 5 돌아올 때는 전치하고 , 빼고 , 순서를 뒤집는다 . 왕복이 맞으면 구현이 맞은 것이다
- 6 배가 기울면 각도 덧셈은 깨진다 — 회전행렬은 필수다
- 7 로컬 평면 좌표를 고도로 그대로 쓰면 안 된다 . 지구는 둥글다

질문 환영합니다 코드 · 문서 : radar-sw-study / Part2_CoordFrames

26

발표 대본

정리 · 약 60초

일곱 가지로 정리하겠습니다.

하나, 좌표계는 "어디에 서서 어느 방향을 기준으로 재는가" 이고 좌표변환은 그 기준을 맞추는 계산입니다. 둘, 변환의 재료는 회전과 평행이동 둘뿐입니다. 셋, 순서는 안테나, 동체, 로컬, ECEF, 위경도이고 건너뛸 수 없습니다. 넷, 회전은 설치 방위와 INS 의 자세 · 위경도가, 평행이동은 레버암과 배의 위치가 공급합니다. 다섯, 돌아올 때는 전치하고 빼고 순서를 뒤집습니다. 왕복이 맞으면 구현이 맞은 것입니다. 여섯, 배가 기울면 각도 덧셈은 깨지고 회전행렬은 필수입니다. 일곱, 로컬 평면 좌표를 고도로 그대로 쓰면 안 됩니다. 지구는 둥글다.

이상입니다. 질문 환영합니다. 코드와 문서는 radar-sw-study 저장소의 Part2_CoordFrames 에 있습니다.

부록 A. 스트랩다운과 짐벌형

"플랫폼 좌표계" 라는 말의 의미가 여기서 갈립니다



짐벌형 (gimbaled)

플랫폼이 물리적으로 수평을 유지한다



"플랫폼 좌표계"가 실제로 존재

스트랩다운 (strapdown)

수평 플랫폼은 계산상으로만 존재 (DCM / 쿼터니언)



현대 함정 INS 는 사실상 전부 이쪽

	짐벌형 (gimbaled)	스트랩다운 (strapdown)
구조	센서가 짐벌 위에 얹혀 있고 모터가 늘 수평 유지	센서가 동체에 직접 고정, 배와 함께 기울고
수평 유지	기계가 물리적으로 한다	계산이 한다 (DCM = Direction Cosine Matrix / 쿼터니언)
"플랫폼 좌표계"	실제로 존재하는 물리적 프레임	소프트웨어 안의 상태값일 뿐
특징	정밀하지만 크고 무겁고 비싸다	현대 함정 INS 는 사실상 전부 이쪽

27

발표 대본

좌표계 · 약 40초

"플랫폼 좌표계" 라는 말의 의미가 여기서 갈립니다.

짐벌형은 센서가 짐벌 위에 얹혀 있고 모터가 늘 수평을 유지합니다. 플랫폼 좌표계가 실제로 존재하는 물리적 프레임입니다. 정밀하지만 크고 무겁고 비쌉니다.

스트랩다운은 센서가 동체에 직접 고정되어 배와 함께 기울고, 수평 유지하는 계산이 합니다. 플랫폼 좌표계는 소프트웨어 안의 상태값일 뿐입니다. 현대 함정 INS 는 사실상 전부 이쪽입니다.

부록 B. 여섯 좌표계 한눈에

좌표계	원점	축	붙은 곳	주 용도
안테나	안테나 면 중심	x 보어사이트 · y 오른쪽 · z 아래	배	표적 측정 (R, Az, El) / 빔 조향 (u, v)
동체	배 무게중심	x 선수 · y 우현 · z 아래	배	탑재 장비의 공통 기준
INS	INS 설치 위치	자이로 · 가속도계 3 축	배	위치 · 자세 · 속도 공급
NED / ENU	배의 현재 위치	N,E,D / E,N,U	바다 위 지점 (회전 안 함)	데이터 처리 (추적 · 필터링)
ECEF	지구 중심	Z 북극 · X 적도 ∩ 그리니치	지구 (자전함)	지구 기준 공통 좌표, 위경도로 가는 다리
ECI	지구 중심	별자리에 고정	없음 (회전 안 함)	위성 · 탄도 표적의 운동 계산
LLA	(적도면 / 그리니치)	위도 · 경도 · 고도	지구	지도 표시, 외부 전달

순서를 건너뛸 수 없습니다. 안테나는 배에 붙어 있고, 배의 방향은 북쪽 기준으로 알려지고, 로컬 원점의 위치는 지구 좌표로만 표현되기 때문입니다.

28

발표 대본

정리 · 약 40초

여섯 좌표계를 표 하나로 모았습니다. 원점, 축, 붙은 곳, 주 용도 순서입니다.

안테나와 동체와 INS 는 배에 붙어 있고, NED 는 바다 위 지점에 붙어 회전하지 않고, ECEF 는 지구에 붙어 자전하고, ECI 는 아무 데도 안 붙어 있고, LLA 는 지구 기준입니다.

중요한 것은 순서를 건너뛸 수 없다는 점입니다. 안테나는 배에 붙어 있고, 배의 방향은 북쪽 기준으로 알려지고, 로컬 원점의 위치는 지구 좌표로만 표현되기 때문입니다.

발표에서는 말하지 않는다. 질문이 들어올 때만 쓴다.

Q1. 왜 ENU 가 아니라 NED 를 썼나?

정보량은 같다. 동체 좌표계가 x 선수, y 우현, z 아래인 FRD 라서, 로컬도 z 를 아래로 두는 NED 를 쓰면 자세각 roll · pitch · yaw 의 부호 규약이 그대로 맞는다. ENU 를 쓰면 축 뒤집는 행렬을 하나 더 끼워야 한다.

Q2. ECEF → LLA 를 왜 Bowring 같은 폐형식으로 안 하고 반복으로 했나?

참고 자료의 순서도가 반복법이라 그대로 따랐다. 초기값은 지구를 구로 본 위도, 고도 0 이고, 위도와 고도 변화가 각각 $1e-12$ rad, $1e-6$ m 아래로 떨어지면 멈춘다. 고도 500 km, 위도 ± 89 도 범위에서 최대 7회면 수렴하는 것을 확인했고 상한은 10회다. 극점 정확히 그 지점은 $\cos(\text{위도})$ 로 나누는 항 때문에 계산이 안 되는데, 함선 운용 범위 밖이라 처리하지 않았다.

Q3. matrixCalcLib 을 쓰면 뭐가 달라지나?

결과는 직접 짠 3x3 과 마지막 자리까지 같다. 라이브러리의 matrix 가 9x9 고정이라 3x3 곱에도 81칸을 초기화하고 656 바이트를 복사하니, 정변환 + 역변환 1회가 약 1 us 에서 약 3 us 로 느려지고 함수당 스택이 4 KB 정도 든다. 스캔당 플롯 1,000개를 처리해도 3 ms 라 이번 범위에서는 문제가 없고, 대신 참고 소스를 그대로 쓴다는 점과 뒤에 올 추적 필터의 6x6 · 9x9 행렬까지 같은 라이브러리로 이어진다는 점이 이득이다. 한 가지 손본 것 없이 원본 그대로 넣었고, Matrix_Identity 가 9x9 로 잡히는 바람에 회전행렬은 Matrix_initialize(3, 3) 로 만든 뒤 0 이 아닌 칸만 채웠다.

Q4. 고도 41.5 m 가 실제 해발고도인가?

Chapter 2 좌표계와 좌표변환 · Janghwan Kim (Harley)

28 / 28

아니다. WGS-84 타원체고다. 해발고도는 여기서 지오이드고를 빼야 하고 한국 근해에서 약 20 ~ 25 m 다. 전시기에 넣길 때 어느 기준인지 ICD 에 명시해야 한다.